Multi-memory approach for Monte Carlo simulation in FPGA

1rd Vinícius P. D. de Oliveira
Dept. Elect. Eng.
Federal University of Juiz de Fora
Juiz de Fora, Brazil
vpdiniz12@gmail.com

2th Pedro H. B. Lisboa
Elect. Eng. Postgraduate Program
Federal University of Rio de Janeiro
Rio de Janeiro, Brazil
https://orcid.org/0000-0002-2931-4105

3nd Ualison R. F. Dias
Elect. Eng. Postgraduate Program
Federal University of Juiz de Fora
Juiz de Fora, Brazil
https://orcid.org/0000-0002-0002-6812

4st Thiago C. A. Paschoalin
Dept. Electroeletronic
Fed. Cent. for Technol. Educ. of Minas Gerais
Leopoldina, Brazil
https://orcid.org/0009-0008-4581-5285

5th Tiago M. Quirino
Dept. Elect. Eng.
Rio de Janeiro State University
Rio de Janeiro, Brazil
https://orcid.org/0000-0002-1799-5222

6th Luciano M. de Andrade Filho
Dept. Elect. Eng.
Federal University of Juiz de Fora
Juiz de Fora, Brazil
https://orcid.org/0000-0003-1792-6793

***Abstract*—Simulating specific data with known distributions is essential in various research fields. To generate values of a random variable, Monte Carlo simulations are widely employed, relying on the Cumulative Distribution Function (CDF) of the process. Modern FPGAs can be leveraged to perform this task by combining a uniform random number generator with a memory that stores the CDF values. However, histogram regions with low occupancy suffer from reduced performance, as the integral contribution in these areas is negligible. To address this, we propose the use of Read-Only Memory (ROM) blocks with varying resolutions, tailored to the occupancy of different regions. As will be demonstrated, this approach enhances data representation without increasing circuit complexity or consuming excessive FPGA resources. The proposed architecture utilizes less than 3% of the target FPGA logic and supports operation at high clock frequencies.**

***Index Terms*—simulation, FPGA, histogram, memory, random variable**

I. Introduction

The generation of high-quality random numbers (RNs) in hardware has become a critical research area due to its essential role in various computationally intensive applications, including financial modeling, Monte Carlo simulations [1], and high-energy physics. While conventional random number generators (RNGs) typically produce uniformly distributed values, numerous applications require non-uniform distributions, with the Gaussian (normal) distribution being among the most widely utilized.

Recent studies have explored multiple strategies for implementing non-uniform RNGs in hardware. The Box-Muller algorithm, a transformation-based method, generates Gaussian-distributed random numbers by applying mathematical transformations to uniform inputs. However, these methods require trigonometric computations, which pose significant challenges in hardware due to their high resource utilization and latency [2]. Rejection-based methods, such as the Ziggurat algorithm, have been optimized for FPGA implementations, achieving high throughput while maintaining statistical quality [3]. Despite their efficiency, these methods may encounter issues with quasi-random sequences, limiting their applicability in some scenarios.

Inversion-based methods, which rely on the inverse cumulative distribution function ICDF, provide an alternative approach by directly mapping uniform random numbers to a target distribution. Efficient ICDF implementation in hardware is nontrivial, often requiring piecewise polynomial approximations or LUTs to achieve acceptable precision. Schryver et. al. [4] proposed a novel architecture that employs floating-point arithmetic to enhance precision and simplify LUT address generation, significantly reducing FPGA resource utilization while maintaining high accuracy in the distribution tails. Lee et. al. [5] introduced an automated methodology for RNG design using hierarchical segmentation and polynomial approximations, enabling the generation of compact, high-performance RNGs for a range of probability distributions.

In the context of Monte Carlo simulations, Banks et. al. [6] compared different PRNG architectures implemented on FPGAs, including Generalized Feedback Shift Registers (GFSRs) and fixed-precision integer multiplication techniques. Their findings indicate that different PRNG approaches exhibit complementary characteristics regarding statistical quality, throughput, and hardware resource consumption. Furthermore, rigorous statistical validation methods, such as the Dieharder test suite, have been employed to assess the reliability of generated random numbers in high-performance computing applications [7]. ThundeRiNG [8] produces multiple independent sequences with low overhead and high statistical quality. A sampling accelerator for robotic tasks was proposed in [9], outperforming CPUs in energy and speed.

A reference table using the ICDF approach is useful to synthesize the data, but there are some limitations depending on how they behave. Increasing the reference table positions helps in fidelity, but the memory usage may extrapolate the

available memory in the desired applications.

This work uses the Monte Carlo simulation, constructing the correspondence table to generate values for a random variable following the desired distribution. The proposed method uses different memories to store some important regions of the Cumulative Density Function (CDF) to maintain the resolution of the curve, without using a very large memory size. Depending on the region, the system will select the best memory to determine the output random variable.

The second section of this paper presents the theory behind Monte Carlo Simulation, highlighting the issues associated with using a limited-space table. Section III introduces the proposed circuit, designed for FPGA implementation, which improves the resolution of specific regions of the histogram. The fourth section details the FPGA board used and outlines the resource utilization. Finally, the last section presents the conclusions of the work.

II. Monte Carlo Simulation

In statistics, a random variable has a Probability Density Function (PDF) that describes its distribution characteristics. Different distributions have their own form. For example, widely known distributions like Gaussian or Poisson have a closed formula linking the random variable and the probability value. The PDF is important to identify how data behaves.

The Cumulative Density Function (CDF) can be used to generate random values according to some known distribution. The CDF is found by integrating the PDF along the random variable. A reference table can be made by dividing the CDF y-axis into equally spaced points and storing the referenced x-axis value [10]. If a uniformly distributed random variable is used as the y-axis value, the resulting values of the x-axis follow the same original distribution of the CDF. This is the basis of the Monte Carlo simulation method [11].

Some distributions have low probability regions and the CDF reflects this with regions where it has a linear curve tendance with an angular coefficient close to zero. If an exponential distribution (f_X) is represented by the Eq. (1) and the respective plot with $\lambda = 1/200$ is presented by Figure 1, and the CDF can be found.

$$f_X(x) = \lambda e^{-\lambda x} \tag{1}$$

The CDF plotted in Figure 2 shows the division of the ordinate axis with 8 positions. It is possible to verify a large region between about 500 and 4000 of the random variable that is not being represented correctly. Plotting both the histograms of 3,000,000 generated numbers using a 1024 correspondence table positions and the original exponential PDF, with the ordinate axis in log-scale, the Figure 3 evidences the problems.

The easiest way to resolve the problem of the resolution is to increase the number of the correspondence table positions. The problem of that approach is the fact that a large memory may be required, which is undesirable for a real-time application with limited resources. Memory size can reach up to 1 MB and even so, may have some resolution problems.

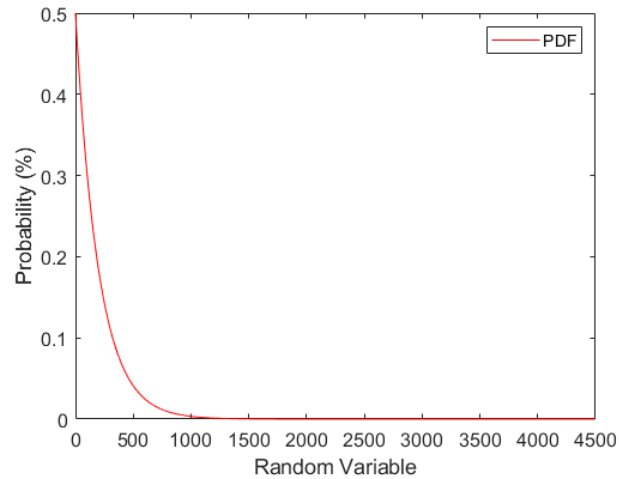

Fig. 1. Histogram of the generated numbers

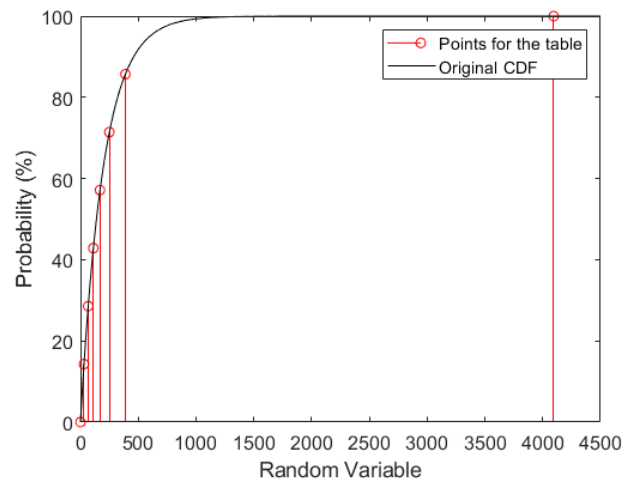

Fig. 2. CDF plot with the detached points for correspondence table

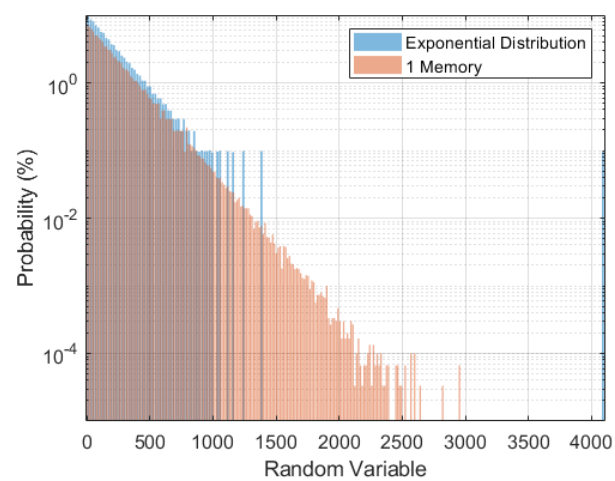

Fig. 3. Histograms of the generated numbers with log-scale y-axis

III. Proposed Solution

The proposed solution for increasing the resolution of the table without spending unwanted resources is to divide the CDF into regions and construct a correspondence table for each region. The tables are preset read-only memories as shown in Figure 4. With this approach, the usage of memories can be optimized, leading to better-generated sequences that follow the original distribution.

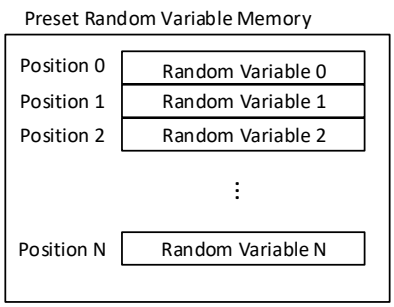

Fig. 4. Preset memory for the correspondence table

Analyzing the previous exponential distribution (Eq. (1)), the CDF can be divided into 3 regions. The Figure 5 shows the extension of each region with its respective memory. The memories can have different sizes to contemplate their region, but in this paper, the same 1024 number of positions is used for all 3 memories.

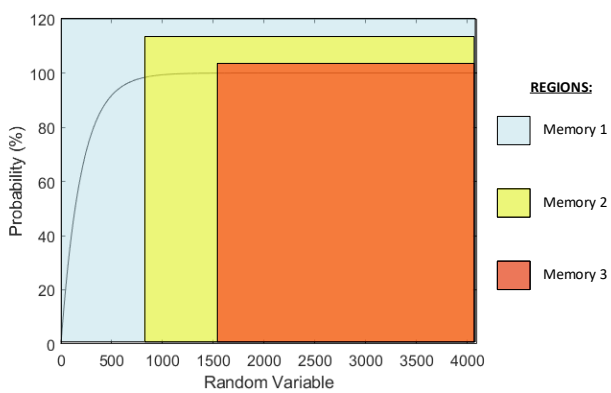

Fig. 5. Histogram divided into regions

The first memory includes all the extension of the CDF. The second begins in a previously defined value where the curve starts the knee of the curve and ends with the same value of the CDF. The last memory includes more advanced values with low probability and includes the rest of the CDF. It is important to detach that the first memory has some values of the y-axis that are shadowing with the second and third memory, while the second memory includes y-axis values of the last memory.

The process for generating the numbers requires the use of a different uniform random number as the input address of each memory. Looking at the first memory, if the address entry is a value that is not shadowing with the second memory, the output generated value is given by the first memory. If the previous condition is false, the input address of the second memory is analyzed and follows the same described process. For y-axis values that do not include values of the last memory, the generated number is given by the second memory. If there is a false condition, the last memory determines the output value. This process is illustrated in the Figure 6. The same idea can be done for other distributions by dividing the CDF into more regions.

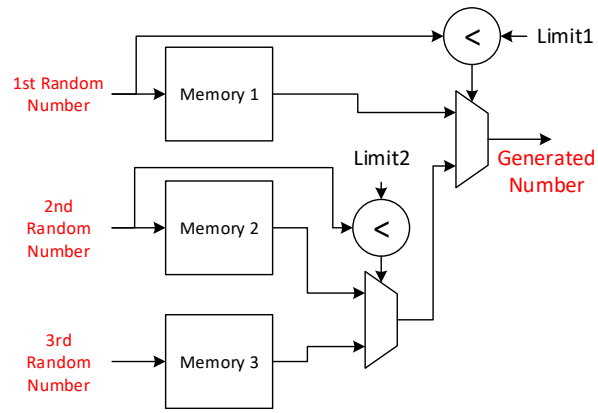

Fig. 6. Block diagram of the FPGA implementation

Generating 3,000,000 numbers with the proposed solution, the new histogram with log-scale can be found in the Figure 7. It is possible to verify that now the resolution has been improved, and it only uses 3072 memory positions in total.

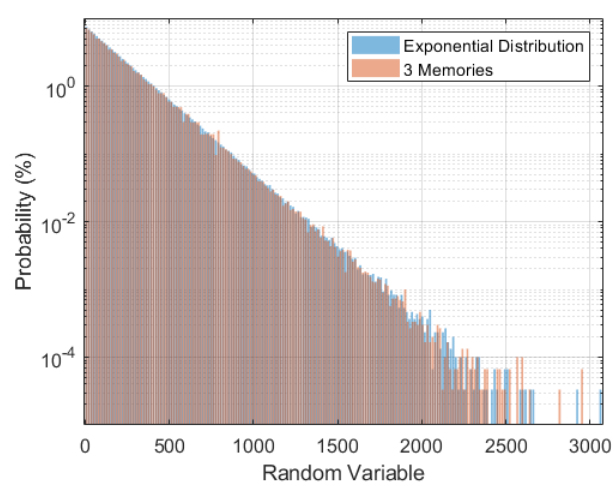

Fig. 7. Histogram of the generated numbers with 3 memories

IV. FPGA Implementation

For the FPGA implementation, it was chosen the DE0-Nano-SoC, a development kit designed by Terasic, and it uses a Cyclone V FPGA family manufactured by Intel. This paper does not include a uniformly distributed random number generator block. These required numbers are external inputs.

The role process for generating numbers according to the desired distribution is represented as a block diagram in the Figure 6. The table I synthesizes the resource usage of the device and shows the maximum clock frequency allowed using the proposed solution.

TABLE I
Resources Usage of DE0-Nano-SoC

Resource	Value	Percentage from Total
Logic Utilization (ALMs)	438	3%
Total Registers	12	-
Total pins	44	14%
Total memory blocks bits	0	0%
Total DSP Blocks	0	0%
Maximum Frequency	450 MHz	-

The described generator uses low resources of the board. Only 3% of ALMs was used and no memory block bits were necessary. This means that all memories are using ALMs to store the original CDF table. The null usage of DSP Blocks is an important feature, because the circuit can produce numbers with complex distributions without any multiplication operation. The maximum frequency is the highest value supported by this FPGA Family.

In this described application, it was not necessary to do any quantization technique for the output numbers generated because the information inside the memory is integer. But, if the values need to have their fractional part represented and it is working with an FPGA that supports only fixed-point numbers, an adequate quantization has to be considered.

V. Conclusion

The CDF of an expected distribution is useful to construct memories that can produce a desired random variable. There are some distributions that can be difficult to characterize without increasing the memory usage. The addition of more memories to express better regions in which the CDF behaves like a linear function with a low angular coefficient can improve the resolution of those areas, generating data much more similar to the original distribution. This leads to a more reasonable use of resources.

To achieve this result, the original memory size was only multiplied by 3, using only 3% of the logic of the Cyclone V FPGA family used. The FPGA can run at full speed with this solution and does not use any DSP blocks.

For future work, the method can be used to generate pulses emulating high-energy particle detectors, facilitating the evaluation of FPGA-based readout systems, as discussed in Quirino et al. [12]. A comparison study between this method and other FPGA-based data generation techniques is essential to determine the best solution for each application.

References

[1] S. Asmussen and P. W. Glynn, *Stochastic Simulation: Algorithms and Analysis*. New York, NY, USA: Springer, 2007.

[2] D. B. Thomas *et al.*, "Gaussian random number generators," *ACM Comput. Surv.*, vol. 39, no. 4, pp. 11-es, 2007.

[3] S. Aniruddhan, "Quadrature generation techniques in CMOS relaxation oscillators," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Seoul, South Korea, 2012, pp. 1375–1378, doi: https://doi.org/10.1109/ISCAS.2012.6271499.

[4] C. de Schryver, D. Schmidt, N. Wehn, E. Korn, H. Marxen, and R. Korn, "A new hardware efficient inversion based random number generator for non-uniform distributions," in *Proc. Int. Conf. Reconfigurable Comput. FPGAs (ReConFig)*, Cancun, Mexico, 2010, pp. 190–195, doi: https://doi.org/10.1109/ReConFig.2010.20.

[5] D.-U. Lee, R. C.-C. Cheung, J. D. Villasenor, and W. Luk, "Inversion-based hardware Gaussian random number generator: A case study of function evaluation via hierarchical segmentation," in *Proc. IEEE Int. Conf. Field Program. Technol. (FPT)*, Bangkok, Thailand, 2006, pp. 33–40, doi: https://doi.org/10.1109/FPT.2006.270388.

[6] S. Banks, P. Beadling, and A. Ferencz, "FPGA implementation of pseudo random number generators for Monte Carlo methods in quantitative finance," in *Proc. Int. Conf. Reconfigurable Comput. FPGAs (ReConFig)*, Cancun, Mexico, 2008, pp. 271–276, doi: https://doi.org/10.1109/ReConFig.2008.38.

[7] R. G. Brown, "Dieharder test suite," 2014. [Online]. Available: https://webhome.phy.duke.edu/~rgb/General/dieharder.php

[8] H. Tan, X. Chen, Y. Chen, B. He, and W.-F. Wong, "ThundeRiNG: Generating multiple independent random number sequences on FPGAs," *IEEE Access*, vol. 9, pp. 120345–120358, 2021, doi: https://doi.org/10.1145/3447818.3461664.

[9] Y. Liu, G. Calderoni, and R. I. Bahar, "Hardware acceleration of Monte-Carlo sampling for energy-efficient robust robot manipulation," *IEEE Transactions on Computers*, vol. 70, no. 2, pp. 287–300, 2021, doi: https://doi.org/10.48550/arXiv.2007.07425.

[10] P. Glasserman, *Monte Carlo Methods in Financial Engineering*. New York, NY, USA: Springer, 2004.

[11] ATLAS Experiment, "Basics of Monte Carlo Simulation," [Online]. Available: https://indico.cern.ch/event/776050/contributions/3237662/attachments/1781016/2897449/MCIntroSaoPaulo.pdf

[12] T. M. Quirino, T. C. A. Paschoalin, G. I. Gonçalves, P. H. B. Lisboa, L. M. de Andrade Filho, and B. S.-M. Peralva, "Online pulse compensation for energy spectrum determination: A pole-zero cancellation and unfolding approach," *Electronics*, vol. 14, no. 3, 2025, doi: https://doi.org/10.3390/electronics14030493.